Presentation Plan

Course: Game Programming
Team: Awake()
Members: Hang Ge, Qiyin Huang, Yanshuo Liu, Yixuan Liu, Xiangtian Ren, Sihan Wang
Total duration: 15 minutes

---

1. Goals for the audience

By the end of the talk, listeners should be able to:

1. Distinguish between `OnTriggerEnter2D`, `OnTriggerStay2D`, and `OnCollisionEnter2D` and explain when each is fired.
2. Identify which functions in a script are Unity event functions and which are helper functions called by them.
3. Describe the first-wins Singleton pattern used by `GameManager` and `UIManager`, and one risk it introduces.
4. Follow a complete event chain from a 2D physics collision all the way to the UI updating on screen.
5. Recognize two concrete event-driven improvements that we propose for the current project.

---

2. Section breakdown

#	Section	Duration	Speaker
1	Opening and post overview	1 min	Member A
2	Concept 1: Physics Callbacks	3 min	Member B
3	Concept 2: Singleton Pattern	2 min	Member C
4	Event chain walkthrough (live trace)	5 min	Member D
5	Why this matters	1.5 min	Member E

6	Improvement 1: Event-driven enemy spawning	1.5 min	Member F
7	Improvement 2: Decoupling score updates with UnityEvent	1 min	Member A
8	Closing reflection and key takeaway	1 min	Member B

---

3. What each section will cover

1. Opening (1 min). State the team, name the two concepts we chose, and explain why we paired Physics Callbacks with the Singleton Pattern instead of treating either one in isolation. Briefly point to the scripts studied (`Damage.cs`, `Health.cs`, `Enemy.cs`, `GameManager.cs`, `UIManager.cs`).

2. Concept 1: Physics Callbacks (2 min). Show all three physics callbacks side by side in `Damage.cs`. Compare them on three dimensions: when Unity fires them, what kind of contact they react to, and which Inspector flag enables each. Connect this to how `Player_Projectile.prefab` enables only `dealDamageOnTriggerEnter`, while `Asteroid` prefabs enable all three.

3. Concept 2: Singleton Pattern (2 min). Walk through the `Awake()` setup in `GameManager` and `UIManager`. Explain "first-wins" behavior: the first component to run `Awake()` claims the static `instance`; later duplicates destroy themselves. Note that `DestroyImmediate(this)` and `Destroy(this)` remove only the script component, not the GameObject. Close with the null-check pattern that every call site uses (`if (GameManager.instance != null)`).

4. Event chain walkthrough (5 min). This is the central section of the talk. A pre-recorded GIF embedded in the slides shows the damage chain animating step by step, so we can narrate at our own pace without switching to the Unity Editor live. The chain we walk through:

```
OnTriggerEnter2D → DealDamage → TakeDamage → CheckDeath → Die
  → (enemy branch) DoBeforeDestroy → AddToScore → GameManager.AddScore → UpdateUI
  → (player branch) GameOver → UIManager.GoToPage(gameOverPageIndex)
```

At each step we highlight whether Unity is calling the function or whether one of our helpers is. We also pause on the branch point (`Die()`) to explain how `Health.cs` decides between the enemy branch and the player branch via `GetComponent<Enemy>()` and tag checks.

5. Why this matters (1.5 min). Make explicit the observation that the chain crosses five scripts but no script knows the full picture. Mention the `collision.gameObject` vs. `gameObject` distinction that the brief warned about and explain why getting it wrong would silently target the wrong object.

6. Improvement 1: Event-driven enemy spawning (1.5 min). Show the current `EnemySpawner.Update()` polling pattern, estimate the wasted frame checks at 60 FPS, and propose the `InvokeRepeating` replacement. Frame this as "letting Unity handle the timing instead of polling it ourselves."

7. Improvement 2: Decoupling score updates with UnityEvent (1 min). Show `GameManager.AddScore()` currently calling `UpdateUIElements()` → `UIManager.instance.UpdateUI()`. Propose adding a `UnityEvent<int> onScoreChanged` so any UI element can subscribe in the Inspector without changing `GameManager`. Honestly note that this decouples `GameManager` from `UIManager` but does not remove the Singleton dependency itself.

8. Closing reflection (1 min). Land on the key insight: in the entire chain we traced, the only function Unity actually called was `OnTriggerEnter2D`. Everything else, including the helpers that do most of the visible work, runs because something earlier in the chain called it. This is the practical meaning of "event-driven" in Unity.

---

4. Visual plan

- All visuals are embedded in the slide deck. We do not switch to the Unity Editor live during the talk.
- The event chain section uses a pre-recorded GIF that animates each step of the chain, so the speaker can narrate at a steady pace.
- Code snippets are short, pre-highlighted, and shown in slides so the audience can read along.
- The improvement section uses two before/after slides, one per improvement, with the changed lines highlighted.